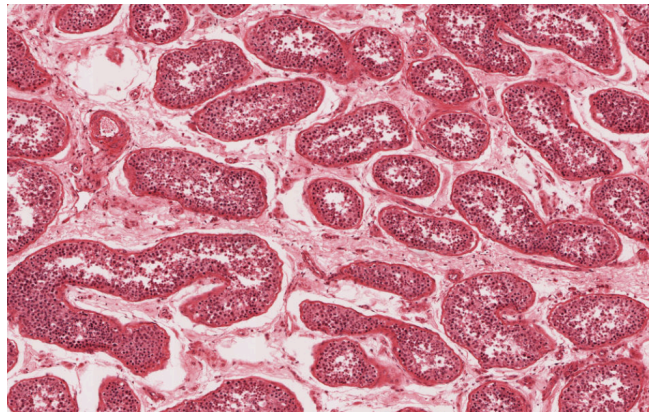




UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INFORMÁTICA Y SISTEMAS
TEMA:

“TIENDA VIRTUAL: UNOFFICIAL ESIS MERCH”



INTEGRANTES:

Marco Antonio Valeriano Mamani 2026-119057

Carles Antonio Vilca Pari 2026-1190

Matías Sebastián Pare Calle 2026-119005

DOCENTE:

Dr. Hugo Manuel Barraza Vizcarra

ASIGNATURA:

Fundamentos de Programación

GRUPO:

“1”

FECHA DE ENTREGA:

15/07/2026

TACNA-PERÚ

2026

1. TÍTULO: “TIENDA VIRTUAL”

2. INTRODUCCIÓN:

El presente proyecto detalla el diseño, estructura e implementación de una aplicación de consola en C++ para la simulación de compras de productos de mercancía universitaria orientada a la Escuela Profesional de Ingeniería de Sistemas (ESIS) de la Universidad Nacional Jorge Basadre Grohmann (UNJBG). La solución utiliza un paradigma estructurado y modularizado mediante la separación de responsabilidades en archivos de cabecera (.h) y de código fuente (.cpp), permitiendo una simulación interactiva con persistencia temporal en memoria.

3. PLANTEAMIENTO DEL PROBLEMA:

Actualmente, los estudiantes y entusiastas de la ESIS que desean adquirir mercancía representativa de la escuela no cuentan con un canal digital centralizado ni interactivo que simule o gestione el catálogo, cálculo de precios con beneficios automatizados (descuentos por volumen) y control de stock de manera dinámica.

Existe la necesidad de diseñar un software eficiente, liviano y modular que sirva como prototipo base para:

- Validar la lógica de negocio de compras múltiples.
- Calcular descuentos automáticos del **33%** a partir de la adquisición de tres productos.
- Controlar la disponibilidad física (stock) de los artículos destacados en tiempo real.

4. OBJETIVOS:

Objetivo General

- Desarrollar una aplicación de consola en C++ que simule el proceso completo de cotización, selección de stock, acumulación en carrito y facturación de artículos de la tienda virtual **UNOFFICIAL ESIS MERCH**.

Objetivos Específicos

- Implementar modularidad de código separando la firma de las funciones de compra (**cabecera.h**) de su comportamiento e implementación lógica (**cabecera.cpp**).
- Programar un algoritmo de descuento por volumen que aplique una reducción del \$33 en el costo final si el usuario agrega 3 o más artículos al carrito.
- Garantizar el control de flujo interactivo del menú principal mediante la lectura de teclado asíncrona (**_getch()**) y validación de entrada por ID de producto.

5. DESARROLLO DE LA PROPUESTA

- Tecnologías de desarrollo (software utilizado, IDE, lenguajes, extensiones, etc.)
 - **IDE / Editor de Texto:** Visual Studio Code (VS Code).
 - **Compilador:** GCC/G++ (a través de la suite MinGW para Windows).
 - **Lenguaje de Programación:** C++ (Estándar C++11 o superior).

- **Extensiones de VS Code:** *C/C++ Extension Pack* (Microsoft) para depuración y autocompletado.
- **Sistema de Control de Versiones:** Git (utilizando ramas de desarrollo como *Marco_rama* y archivo *.gitignore* configurado para ignorar **.exe*).
- **Librerías del Sistema Utilizadas:**
 - **<iostream>**: Flujos de entrada y salida de datos por consola (*cin/cout*).
 - **<conio.h>**: Captura de eventos del teclado en consola (*_getch()*).
 - **<string>**: Manejo de cadenas de texto dinámicas.
 - **<cstdlib>**: Limpieza de pantalla (*system("cls")*).

○ Descripción de los interesados y usuarios

■

Rol	Descripción	Relación con el Sistema
Cliente / Estudiante ESIS	Usuario final de la aplicación.	Busca visualizar artículos, agregarlos al carrito de compras, verificar si califica a descuentos y concretar la compra utilizando su saldo disponible.
Administrador / Desarrollador	Encargado del mantenimiento de software.	Modifica los precios, actualiza el inventario inicial en la estructura del código y analiza la eficiencia del algoritmo de cobro.

○ Requerimientos funcionales

- **RF-01 (Visualización de Catálogo):** El sistema debe listar de forma tabular el nombre, ID único, precio y cantidad disponible de cada artículo de la tienda.
- **RF-02 (Gestión de Carrito):** El sistema debe permitir añadir artículos al carrito ingresando su ID único, descontando inmediatamente una unidad del inventario temporal.
- **RF-03 (Validación de Fondos):** El usuario no podrá agregar un producto al carrito si el costo total del carrito supera el dinero disponible en su cartera virtual (*Bolsillo_derecho*).
- **RF-04 (Aplicación de Descuento):** Si el carrito contiene 3 artículos o más, el sistema aplicará un descuento equivalente al 33% del total acumulado.
- **RF-05 (Proceso de Pago Secreto):** La compra del carrito acumulado se procesará de forma exclusiva al ingresar el código o ID especial *"!buy"*.

- **RF-06 (Historial de Mochila):** El sistema debe mostrar el listado detallado de los artículos adquiridos (unidades compradas y gasto individualizado) comparando la cantidad de stock inicial restante.

- Arquitectura de software

- Lista de datos estructurados utilizados (detallar campos y sus significados)

El programa define una sola estructura principal (**struct**) en **main.cpp** para modelar el catálogo de artículos:

Estructura: Productos

Almacena la información de los 5 artículos de la tienda de forma indexada a través de arreglos paralelos de tamaño fijo (5):

Campo	Tipo de Dato	Propósito	Ejemplo de Dato
nombre[5]	string (Arreglo)	Almacena el nombre comercial identificativo de los productos.	"Teclado mecanico"
ID[5]	string (Arreglo)	Clave alfanumérica única requerida para buscar y comprar un artículo.	"0x45234"
precio[5]	double (Arreglo)	Costo monetario unitario de cada artículo en dólares (\$).	175.5
cantidad[5]	int (Arreglo)	Stock o unidades remanentes disponibles en tienda (Inicia en 5).	5

- Lista de funciones clasificadas por áreas.

Área de Interfaz y Control de Flujo (**main.cpp**)

- **main()**: Función principal encargada de inicializar las variables globales del cliente. **Ahora incluye la subrutina inicial para leer y validar el dinero del bolsillo**. También gestiona el bucle infinito del menú, captura pulsaciones de teclado numéricas y renderiza las pantallas de "Inicio", "Tienda" y "Tu Mochila".

Área de Transacción Comercial (**cabecera.h** / **cabecera.cpp**)

- **comprar(double &bolsillo, double &carrito, int &articulos)**:
Procesa el pago final del carrito acumulado.
 - **Parámetros**: Recibe referencias en memoria (&) para poder editar directamente el estado financiero del usuario dentro de **main()**.
 - **Lógica**: Calcula la reducción del bolsillo con o sin descuento del 33%, y formatea el valor del carrito y cantidad de artículos de vuelta a \$0\$.
- Diagramas de flujo y descripción de las funciones más importantes. (Puede usar [Zinjal](#) para generar diagramas)
- Capturas de menús de opciones más importantes.

- **Menú Principal (Flujo base)**

Muestra el encabezado interactivo y espera una pulsación directa sin necesidad de pulsar *Enter*:

6. CONCLUSIÓN:

Modularidad Cumplida: La división estructural del código facilitó un entorno de pruebas limpio. Al aislar la lógica matemática en **cabecera.cpp**, se resguarda el motor de cálculo financiero del flujo visual del usuario.

Control de Memoria Eficiente: El uso de herencia de datos por referencia (**double &**) demostró ser crucial para el control del presupuesto del cliente, evitando duplicidad innecesaria de variables y garantizando consistencia en el estado del programa.

Funcionamiento del Descuento: El algoritmo cumple de manera exacta con las políticas de promoción propuestas para la tienda, aplicando adecuadamente la reducción del cobro si el cliente acumula tres artículos o más antes de ingresar el código secreto **"!buy"**.